DAY 3 · CLOSING LECTURE · ~30 MINUTES

The Quartermaster's Manifest

Everything this island can teach — and the idea that makes it possible

A manifest is the ship's cargo list. It is also a Kubernetes file. Today, both meanings at once.

Two questions before we close

You spent today **building**: a Helm chart, a GitOps pipeline, an ArgoCD that heals itself.

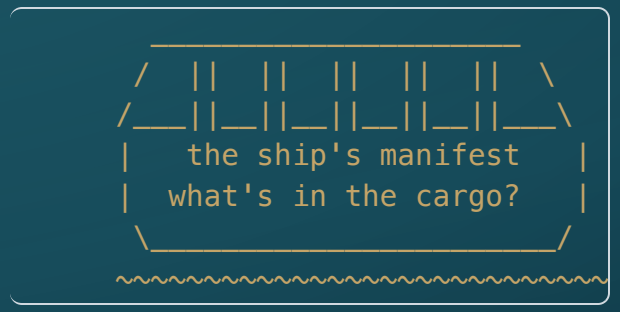
Step back and ask:

1. **What can you actually *teach* on a platform like this?** — far more than DevOps. (Part I)
2. **What is this thing you've been building all week even called?** — it has a name, a literature, and a career path. (Part II)

> *The tools were the vehicle. These two answers are the souvenir you take home.*

PART I

The Hold



> *"Open the hold. Let them see everything we're carrying."*

One cluster. Every subject.

- Kubernetes is filed under "DevOps." That is a **category error.**
- What you really have is a **lab-delivery platform**: a way to hand any student an identical, isolated, disposable environment over a URL.
- The subject inside that environment can be *anything* — Python, AI, networking, C, databases, security.
- Everything on the next few slides runs on the **same island you already stood up** — no new infrastructure, no IT ticket.

Find your subject on the shelf. The point is: it's already there.

Python · Data Science · Analytics

Platform	What it lets you teach
JupyterHub (Zero-to-JupyterHub)	A private notebook server *per student*, spawned on demand. The classroom standard at Berkeley & Brown.
Binder / BinderHub	Turn any Git repo into a live notebook over a URL — zero install.
Spark / Dask / Ray on K8s	Distributed data processing; "big data" without a data center.
Airflow	Data-engineering pipelines and DAG orchestration.

> *The same GitOps loop you ran in Lab 02 is how a JupyterHub gets delivered: one chart in Git, every student's notebook server in sync.*

AI · Machine Learning · LLMs

Platform	What it lets you teach
Ollama + Open WebUI	Self-hosted LLMs, entirely in the room — the Boatswain you've used all week. Prompt engineering, RAG, local inference.
Kubeflow	End-to-end MLOps: notebooks, training pipelines, model registry.
KServe / Seldon	Model *serving* — deploy a model as a live API and teach inference at scale.
vLLM + GPU scheduling	High-throughput LLM inference; share one GPU across many students (time-slicing / MIG).

> *The AI never has to leave the building — no per-seat API bill, no data egress.*

Web · JavaScript · npm · Full-Stack

Platform	What it lets you teach
code-server (VS Code in a browser)	A full IDE per student over a URL — no laptop setup, identical for all 30.
The 3-tier app you built today	Frontend / backend / database, real services, real routing.
Preview environments (vCluster + ArgoCD)	A live, isolated deploy per pull request — teach real review workflows.
Gitea Actions / Tekton	npm build + test pipelines that run in-cluster.

> *"Works on my machine" dies here: the machine is the cluster, and everyone has the same one.*

Systems · C / C++ · Operating Systems

Platform	What it lets you teach
code-server + toolchain image	A pinned compiler/build environment, identical for everyone — no "which gcc?"
Reproducible build containers	The exact same libc, headers, and flags for every student, every term.
KubeVirt	*Full virtual machines* as cluster objects — kernel work, drivers, OS internals, root access, safely.
Argo / Tekton pipelines	CI for compiled languages: build matrices, unit tests, artifacts.

> *When a student melts their VM: delete it, redeploy, back in a minute.*

Networking

Platform	What it lets you teach
Clabernetes / Containerlab	Virtual **commercial router** topologies — Nokia SR Linux, Arista, Cisco. BGP, OSPF, EVPN. Replaces GNS3 / EVE-NG.
Cilium + Hubble	eBPF networking, NetworkPolicy, live traffic flow maps.
Istio / Linkerd	Service mesh: traffic shaping, mTLS, retries, canaries.
KubeVirt	Windows Server, Active Directory, DHCP/DNS — on the *same* fabric as the containers.

> *Clabernetes is one item on this shelf — the heavy-hardware-replacement that used to need a rack of gear.*

DevOps · SRE · Cloud-Native

Platform	What it lets you teach
ArgoCD / Flux	GitOps — what you did today.
Helm / Kustomize	Templating and configuration — also today.
Prometheus / Grafana / Loki	Metrics, dashboards, logs — observability from scratch.
Chaos Mesh / LitmusChaos	Resilience and failure testing — *tomorrow's pirate.*
Harbor + Trivy + cosign	Registry, image scanning, signing — software supply-chain security.

> *This is the "home" discipline — but notice it's only* **one column** *of the manifest.*

Databases · Security · and the rest

Platform	What it lets you teach
CloudNativePG & DB operators	Real high-availability Postgres / MySQL / Mongo — actual DBA skills, with failover.
MinIO	S3-compatible object storage — cloud storage patterns, locally.
Falco + **Kyverno / OPA**	Runtime threat detection and policy-as-code — defensive security.
Vault	Secrets management done properly.
KubeVirt sandboxes	Isolated VMs for malware analysis, CTFs, red-team labs.

The five superpowers (why *any* subject wins)

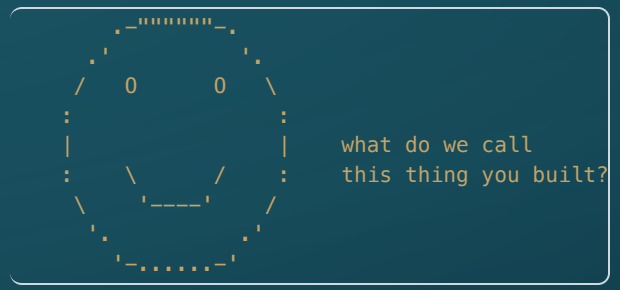
Whatever you teach, the platform gives you the same five things:

1. **One URL, identical environment** — 30 students, zero local setup, no "works on mine."
2. **Push to Git → everyone updates** — distribute a lab change to the whole class instantly.
3. **Isolated & disposable** — break it freely, reset in a minute.
4. **Self-hosted & private** — air-gapped if you want; no per-seat SaaS, data stays in the room.
5. **The system state *is* the grade** — auto-gradeable labs (you'll see this tomorrow).

> *These are subject-independent. That is the whole point.*

PART II

The Idea Underneath



> "*You've been speaking prose your whole life without knowing it.*"

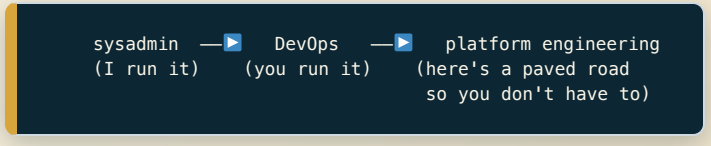
You've been a platform engineer all week

- A platform team builds a **self-service environment** so *developers* can do their job without becoming infrastructure experts.
- Swap two words:

Industry	Your classroom
Platform team	**You**
Developers	**Your students**
Internal Developer Platform	**The lab**
"Ship features, don't fight infra"	"Learn the subject, don't fight setup"

- The problem is *identical*: many people need a working environment; none of them should have to build it from scratch.

How we got here



- **DevOps** said: developers should own their infrastructure. Good in spirit — but it dumped a *mountain* of cognitive load on people who just wanted to ship.
- **Platform engineering** is the correction: give people a **supported, opinionated path** so they get self-service *without* having to learn everything underneath.
- You live this every term. "*Set up your own environment*" is the DevOps-era overload. **The lab is the paved road.**

The vocabulary: IDP

INTERNAL DEVELOPER PLATFORM

- The **self-service layer** that sits between a person and the messy infrastructure.
- A developer (student) asks for *what they want* — a notebook, a database, a cluster — and gets it, without filing a ticket or reading a wiki.
- It **abstracts the complexity away.** They don't see Helm, ArgoCD, namespaces — they see "my environment is ready."

> *In your class, the IDP is the lab. Today you built a working one.*

The vocabulary: the Golden Path

- The **one supported, opinionated, well-lit route** through a common task.
- Spotify named it the **"Golden Path."** Netflix calls it the **"Paved Road."**
- Not a cage — the *easy* path. Anything-goes overwhelms; forced-standardization rebels. The golden path is the sweet spot in between.
- **You already build these:** a starter repo, a setup script, "here's the one way we do it in this course." Now you have the name.

> *A golden path is how one of you supports thirty of them without burning out.*

The mindset: platform as a *product*

- A platform has **users**, and users **vote with their feet.**
- If your lab is painful, students route around it — work on their laptops, fall behind, drop the tool.
- So you design for **adoption, not mandate**: make the supported path the *easiest* path.
- Start with the **thinnest viable platform** — the smallest thing that actually helps — and grow it from feedback.

> *The same instinct that makes a good course makes a good platform: meet the user where they are.*

The goal: reduce cognitive load

- Every learner has a **fixed budget** of working memory.
- Spend it on *the subject* — BGP, recursion, gradient descent — not on *kubeconfig and pip*.
- A platform's job is to **absorb the incidental complexity** so the scarce attention goes to the actual learning.
- This is the *Team Topologies* insight, and it is just good pedagogy with an engineering name.

> *Yesterday's worry — "abstraction stacking" — is cognitive load. The platform is how you pay it down.*

Where any lab sits: the maturity ladder

The CNCF Platform Engineering Maturity Model, read as an educator:

Level	In industry	In your classroom
1 · Provisional	ad-hoc, manual	"Set it up on your laptop." Snowflakes, *works-on-mine.*
2 · Operational	a shared platform exists	One JupyterHub / code-server everyone uses.
3 · Scalable	self-service & repeatable	GitOps-distributed labs, per-student isolation, rebuildable from a repo.
4 · Optimizing	a product, measured	A self-service catalog; students provision their own; you measure & iterate.

> *Most college labs live at* **Level 1.** *Getting to* **Level 2** *is a giant leap — and you can do it alone.*

The pocket test: four questions

Hold any lab you run up to these four questions:

1. **Self-service** — can a student get a working environment *without me?*
2. **Golden path** — is there *one obvious, supported* way to do it?
3. **Reset** — if they break it, can they get back to clean *fast?*
4. **Distribution** — when I improve the lab, does *everyone* get the update?

> *Four "no"s is Level 1. Four "yes"es is the platform you used today.*

What to do Monday morning

- **Don't build Backstage.** Don't boil the ocean.
- Pick **one** painful environment in **one** class — the one where setup eats the first week.
- **Containerize it once. Deliver it over a URL** (code-server or JupyterHub).
- That's a golden path. That's Level 2. That's the whole game, started.

> *The smallest paved road beats the grandest plan that never ships.*

The tools to go look up

Named, not taught — your map for after the seminar:

Tool	What it does	When you'd reach for it
Backstage (CNCF)	Developer portal / catalog	A "front door" for your courses & labs
Crossplane (CNCF)	Provision infra via the K8s API	Students request environments declaratively
ArgoCD	GitOps delivery	Push a change → everyone gets it (you did this today)
vCluster	A real cluster per user	Self-service, isolated clusters (tomorrow's demo)
Port / Humanitec	Commercial IDPs	If you'd rather buy the portal than build it

Read these on the flight home

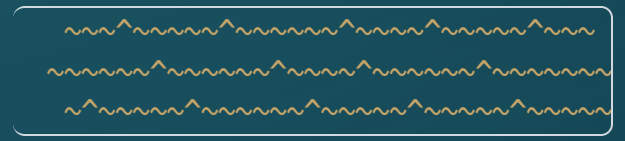
- **CNCF Platform Engineering Maturity Model** — the framework on the ladder slide. `tag-app-delivery.cncf.io`
- **CNCF Platforms Working Group** — the community defining this field. `platforms.cncf.io`
- **Team Topologies** (Skelton & Pais) — the book behind "reduce cognitive load."
- **internaldeveloperplatform.org** — vendor-neutral IDP reference.
- **platformengineering.org** — community, blog, the golden-path writing.

Instructor Superpower

YOU ARE THE PLATFORM TEAM FOR YOUR STUDENTS

- All week you learned the platform. **Platform engineering is the discipline of building it on purpose.**
- Every hour you spend paving a road is an hour *thirty* students don't spend lost in setup.
- You don't wait for IT to offer a capability — you have just seen that you can **build the environment yourself**, for a lesson, a course, or a whole program.
- The job was never "teach Kubernetes." The job is **clearing the path to the thing you actually teach.**

The hold is open.



Tomorrow morning we take three items off this shelf — vCluster, KubeVirt, and Chaos — and go deep.

Then the Pirate comes.